Colas de mensajes y Transactional Outbox

Nov 11, 2026 · @Lucas

Datos del trabajo

Institución	Universidad Tecnológica Nacional – Facultad Regional Resistencia
Carrera	Ingeniería en Sistemas de Información
Asignatura	Desarrollo de Software – 2026
Trabajo	Investigación y aplicación de conceptos de Desarrollo de Software
Grupo	Grupo 1 (Módulo M2 – Clientes del Trabajo Práctico Integrador)
Temas asignados (Par 04)	Tema 2: Colas de mensajes y procesamiento asíncrono · Tema 27: Patrón Transactional Outbox

Integrante	Legajo
Brandt, Carina Noemí	24672
Caminero, Valentina	28024
Giraudo, Juan Manuel	24332
Nuñez, Lucas Emanuel	23959
Paluch Ruberto, Julián	27229
Sonza, Matías Javier	26573

1. Introducción y objetivo

Las colas de mensajes permiten que un servicio responda rápido y delegue el trabajo lento a otro proceso. El patrón Transactional Outbox garantiza que ese mensaje se publique siempre que el cambio en la base de datos se haya confirmado, y nunca si no se confirmó. Los dos temas son complementarios: el primero introduce la comunicación asíncrona y el segundo resuelve su punto más frágil, el momento de publicar.

Las preguntas guía planteadas por la cátedra son:

- **Tema 2:** ¿Cómo desacoplar una operación lenta de la respuesta HTTP mediante una cola y qué sucede si el consumidor deja de funcionar?
- **Tema 27:** ¿Cómo garantizar que una modificación en la base de datos y la publicación de un evento no queden inconsistentes ante una falla?

El objetivo es explicar ambos conceptos, analizar cuándo conviene usarlos y cuándo no, relacionarlos con la plataforma de movilidad urbana del Trabajo Práctico Integrador (TPI) y proponer una demostración que los combine. El TPI ya los anticipa: el RNF-10 exige RabbitMQ para los flujos asíncronos, el RF-8.8 pide que las notificaciones se procesen desde una cola y el RNF-09 exige que los consumidores toleren mensajes repetidos.

> **Motor de base de datos: MongoDB en el TPI, PostgreSQL en la demo.** El módulo M2 del TPI usa MongoDB. La demostración de este trabajo es un proyecto independiente y usa PostgreSQL, porque el patrón Transactional Outbox nació en bases relacionales y ahí sus garantías (una transacción con `BEGIN` y `COMMIT` sobre dos tablas) se ven de forma más directa. Todos los ejemplos de código usan Python, FastAPI, PostgreSQL (librería psycopg) y RabbitMQ (librería pika). Las secciones 4.7 y 6.3 explican qué cambia al aplicar el patrón en MongoDB, el motor del TPI.

2. Fundamentos previos

Dos conceptos sostienen todo lo que sigue: qué significa cada respuesta HTTP y qué garantiza una transacción.

2.1 HTTP y códigos de estado

HTTP define cómo un cliente (la app o el navegador) envía una solicitud a un servidor (la API) y cómo este responde. Cada solicitud tiene un método (`GET` para consultar, `POST` para crear, `PUT` / `PATCH` para modificar, `DELETE` para eliminar), una ruta (`/viajes`), cabeceras con metadatos (`Content-Type: application/json`) y, opcionalmente, un cuerpo en JSON. La respuesta empieza con un código de estado de tres dígitos:

Código	Significado	Relevancia para estos temas
`200 OK` / `201 Created`	El procesamiento terminó en ese mismo instante	Respuesta de una operación síncrona
`202 Accepted`	La solicitud es válida y quedó registrada, pero el procesamiento se hará después	Respuesta típica al encolar un trabajo

Código	Significado	Relevancia para estos temas
`4xx`	Error del cliente: datos inválidos (`400`), recurso inexistente (`404`)	La API valida antes de encolar
`5xx`	Error del servidor: falla interna (`500`), dependencia no disponible (`503`), demora excesiva de un intermediario (`504`)	Lo que el procesamiento asíncrono busca evitar

Una API web, por ejemplo una aplicación FastAPI, es un proceso que escucha un puerto (en M2, el 8000) y asocia cada combinación de método y ruta a una función, llamada *endpoint*.

2.2 Transacciones: todo o nada

Una transacción agrupa varias escrituras para que se confirmen todas juntas (*commit*) o se descarten todas (*rollback*). Es la propiedad de **atomicidad**, la "A" de ACID. En una base relacional se delimita con `BEGIN` y `COMMIT`; si alguna instrucción falla o el proceso se cae antes del `COMMIT`, el motor deshace todo lo hecho desde el `BEGIN`. Esta garantía la da el motor de base de datos mediante su registro de transacciones (en PostgreSQL, el *Write-Ahead Log*; en MongoDB, el *journal* y el *oplog*).

La limitación clave para este trabajo es que **una transacción abarca un solo motor de base de datos**. No existe un `COMMIT` común entre PostgreSQL y RabbitMQ, que son dos servidores distintos. De esa limitación nace el problema que resuelve el Transactional Outbox.

3. Colas de mensajes y procesamiento asíncrono

Una cola de mensajes es un intermediario que guarda pedidos de trabajo hasta que otro proceso los toma y los procesa. Quien envía no espera el resultado: deja el mensaje y sigue. Esto es comunicación **asíncrona**, en oposición a una llamada REST **síncrona**, donde quien llama queda bloqueado hasta recibir la respuesta.

3.1 El problema del procesamiento síncrono

Cuando un cliente envía `POST /viajes`, la conexión queda abierta hasta que el endpoint termina. Si el endpoint incluye una operación lenta (buscar conductores, llamar a una pasarela de pagos, generar un PDF), esa demora se traslada entera a la respuesta. Esto produce tres fallas:

1. **Timeouts:** los proxies y balanceadores cortan las conexiones que superan un límite, habitualmente cercano a 60 segundos, y el cliente recibe `504 Gateway Timeout` aunque el trabajo siga corriendo.
2. **Saturación de recursos:** si cada tarea usa 100 MB de memoria y llegan 50 pedidos en el mismo segundo, la API intenta procesarlos a la vez: 50 × 100 MB = 5000 MB. El servidor puede quedarse sin memoria y el sistema operativo terminar el proceso.
3. **Pérdida del trabajo en curso:** si la API se reinicia en el segundo 8 de una tarea de 10 segundos, lo que estaba en memoria se pierde y la operación queda inconclusa.

Además, si el servicio que hace el trabajo lento está caído, la operación completa falla.

3.2 Arquitectura: productor, broker y consumidor

Componente	Función	En la demo
Productor	Recibe la solicitud HTTP, empaqueta los datos de la tarea en un mensaje JSON y responde `202 Accepted` sin ejecutar el trabajo lento	La API de Viajes (FastAPI, puerto 8000)
Broker	Servidor que recibe, guarda y entrega los mensajes	RabbitMQ (puerto 5672)
Exchange	Punto de entrada del broker que decide a qué cola va cada mensaje	Exchange por defecto o `viajes`
Cola	Almacén de mensajes pendientes; entrega en orden de llegada (FIFO)	`asignacion.viaje-solicitado`
Consumidor (worker)	Proceso independiente que no atiende HTTP: toma mensajes y ejecuta el trabajo lento	`worker.py`
Acknowledgement (ack)	Confirmación del consumidor de que terminó de procesar un mensaje	`basic_ack`

El orden FIFO vale para la entrega, pero no garantiza el orden de procesamiento: con varios consumidores o con mensajes reentregados, dos mensajes pueden terminar en otro orden.

Este esquema es el patrón Asynchronous Request-Reply: la respuesta `202` incluye un encabezado `Location` con la dirección donde el cliente consulta el estado del trabajo.

3.3 Ventajas aun con un solo worker

Una cola mejora el sistema aunque haya un único consumidor procesando de a una tarea:

1. **Respuesta inmediata:** si llegan 50 pedidos a la vez, la API deja 50 mensajes en la cola y responde `202` a todos en milisegundos.
2. **Nivelación de carga:** 50 mensajes JSON ocupan pocos kilobytes en el broker. Con `prefetch_count=1`, RabbitMQ entrega al worker un solo mensaje a la vez, así que el consumo de memoria se mantiene constante aunque haya un pico (Queue-Based Load Leveling).
3. **Aislamiento:** mientras el worker usa la CPU en una tarea pesada, la API sigue libre para responder consultas livianas.
4. **Escalabilidad horizontal:** si la cola crece, se inician más workers sobre la misma cola sin tocar la API. RabbitMQ reparte los mensajes entre ellos por turnos (Competing Consumers).

3.4 Qué pasa si el consumidor deja de funcionar

RabbitMQ solo borra un mensaje cuando recibe el ack del consumidor (documentación de RabbitMQ). Cada mensaje pasa por dos estados internos: **Ready** (listo para entregar) y **Unacked** (entregado, esperando confirmación).

Escenario	Comportamiento
El worker está detenido cuando llegan los mensajes	La API sigue respondiendo `202`. Los mensajes quedan en estado Ready. Si la cola es durable (`durable=True`) y los mensajes persistentes (`delivery_mode=2`), se guardan en disco y sobreviven a un reinicio del broker
El worker se cae a mitad de una tarea	El mensaje estaba en Unacked. Al cortarse la conexión del worker, RabbitMQ lo vuelve a Ready y lo entrega de nuevo con la marca `redelivered`
El worker vuelve a iniciarse	Recibe todos los mensajes en Ready y completa las tareas pendientes sin pérdida

La conclusión responde la pregunta guía: **si el consumidor deja de funcionar, el trabajo no se pierde, se demora.** El requisito es usar ack manual (`auto_ack=False`); con ack automático RabbitMQ borra el mensaje al entregarlo y un worker caído lo perdería.

```
# worker.py (versión básica; la versión idempotente está en la sección 5.3)
def procesar(canal, metodo, propiedades, cuerpo):
    mensaje = json.loads(cuerpo)
    asignar_conductor(mensaje["viajeId"])  # operación lenta
    # Recién ahora RabbitMQ puede borrar el mensaje
    canal.basic_ack(delivery_tag=metodo.delivery_tag)

canal.queue_declare(queue="asignacion.viaje-solicitado", durable=True)
canal.basic_qos(prefetch_count=1)  # un mensaje a la vez
canal.basic_consume(queue="asignacion.viaje-solicitado",
                    on_message_callback=procesar, auto_ack=False)
canal.start_consuming()
```

3.5 Entrega "al menos una vez" y otras características

- **Duplicados:** si el worker termina el trabajo pero se cae antes de enviar el ack, el mensaje se reentrega y se procesa dos veces. RabbitMQ garantiza entrega **al menos una vez**, no exactamente una vez. Por eso el consumidor debe ser **idempotente** (sección 5.3).
- **Confirmación del publicador (*publisher confirms*):** con `confirm_delivery()` activado, el broker le confirma al productor que guardó el mensaje. Sin esa confirmación, el productor no sabe si el mensaje llegó.
- **Dead Letter Queue (DLQ):** los mensajes rechazados o que superan un límite de reintentos se desvían a una cola aparte para analizarlos, en lugar de trabar la cola principal (Dead Letter Exchanges).

3.6 Beneficios y limitaciones

Beneficios	Limitaciones
Respuesta rápida aunque el procesamiento tarde	**Consistencia eventual:** el resultado no está disponible al instante
Tolerancia a la caída del consumidor	Duplicados posibles: exige consumidores idempotentes
Absorbe picos de carga	Sin orden garantizado de procesamiento con varios consumidores

Beneficios	Limitaciones
Escala agregando workers	Un componente más para desplegar y monitorear
Desacopla productor y consumidor	Depuración más difícil: hay que correlacionar logs por identificador (RNF-14)

La limitación más importante para este trabajo es que **la cola protege el mensaje solo desde que entra al broker**. Si el productor guarda el dato en su base y se cae antes de publicar, el mensaje nunca llega. Ese problema lo resuelve el tema siguiente.

4. Patrón Transactional Outbox

El Transactional Outbox guarda el evento a publicar en la misma base de datos y en la misma transacción que el cambio de negocio; un proceso aparte, el *relay*, lo publica después en el broker (microservices.io). Así, el evento existe si y solo si el cambio se confirmó.

4.1 El problema: la escritura dual

Cuando la API recibe `POST /viajes` tiene que hacer dos cosas: guardar el viaje en su base de datos y publicar un mensaje para que el worker lo procese. PostgreSQL y RabbitMQ son servidores distintos y no comparten una transacción, así que escribir en ambos desde el endpoint (*dual write*) falla en cualquier orden.

Falla 1: guardar primero y publicar después.

```
@app.post("/viajes", status_code=202)
def solicitar_viaje_inconsistente(datos: SolicitudViaje):
    viaje_id = str(uuid.uuid4())
    with psycopg.connect(DSN) as conexion:
        conexion.execute("INSERT INTO viajes (id, estado) VALUES (%s,
'SOLICITADO')",
                         (viaje_id,))
    # Al salir del bloque se hizo COMMIT: el viaje ya quedó guardado
    # Si RabbitMQ está caído o el proceso se corta acá, el mensaje nunca sale
    canal.basic_publish(exchange="", routing_key="asignacion.viaje-
solicitado",
                        body=json.dumps({"viajeId": viaje_id}))
```

El viaje queda guardado en estado `SOLICITADO`, pero el mensaje nunca entra a la cola. Aunque RabbitMQ vuelva minutos después, no tiene nada sobre ese viaje y el worker nunca lo procesa: **evento perdido**.

Falla 2: publicar primero y guardar después. El mensaje sale hacia RabbitMQ y, un instante después, la escritura en la base falla (validación, conflicto o caída). El viaje no existe, pero el worker recibe el mensaje e intenta procesarlo: **evento fantasma**. Publicar dentro de la transacción, antes del commit, tiene el mismo problema si el commit falla.

La alternativa clásica, la transacción distribuida en dos fases (2PC), coordina ambos sistemas, pero RabbitMQ no la soporta y además bloquea recursos y acopla los servicios.

4.2 La solución: bandeja de salida en la misma base

Outbox significa bandeja de salida. El patrón elimina la escritura directa de la API en RabbitMQ y la reemplaza por dos piezas.

Pieza 1: una tabla `outbox` y una sola transacción. En la misma base donde está la tabla `viajes` se crea una tabla `outbox` (su definición está en la sección 4.6). Cada fila tiene un identificador único del evento, su tipo, el contenido (*payload*) en JSON, la fecha y el estado (`PENDIENTE` o `PUBLICADO`). La API inserta el viaje y el evento en la misma transacción y no se conecta a RabbitMQ.

```
@app.post("/viajes", status_code=202)
def solicitar_viaje(datos: SolicitudViaje):
    viaje_id = str(uuid.uuid4())
    evento_id = str(uuid.uuid4())      # lo usa el consumidor para deduplicar
    payload = {"viajeId": viaje_id, "clienteId": datos.cliente_id}
    # BEGIN implícito; al salir del bloque, COMMIT. Si hay una excepción,
ROLLBACK de las dos filas
    with psycopg.connect(DSN) as conexion:
        conexion.execute(
            "INSERT INTO viajes (id, estado, origen, destino) VALUES (%s,
'SOLICITADO', %s, %s)",
            (viaje_id, datos.origen, datos.destino))
        conexion.execute(
            "INSERT INTO outbox (id, tipo, payload) VALUES (%s,
'ViajeSolicitado', %s)",
            (evento_id, Jsonb(payload)))
    return JSONResponse({"viajeId": viaje_id, "estado": "SOLICITADO"},
status_code=202,
                        headers={"Location": f"/viajes/{viaje_id}"})
```

Como las dos escrituras están en la misma transacción, se guardan las dos o ninguna. Y como la API no habla con RabbitMQ, **si el broker está caído la API sigue respondiendo `202` con normalidad**.

Pieza 2: el relay. Un proceso independiente consulta periódicamente los eventos `PENDIENTE`, los publica y los marca como `PUBLICADO`. Esta técnica se llama *polling publisher*.

```
# relay.py
COLA = "asignacion.viaje-solicitado"

def conectar():
    conexion = pika.BlockingConnection(pika.ConnectionParameters("rabbitmq"))
    canal = conexion.channel()
    canal.confirm_delivery()                    # el broker confirma cada
publicación
    canal.queue_declare(queue=COLA, durable=True)
    return canal

canal = None
while True:
    try:
        if canal is None or canal.is_closed:
            canal = conectar()                  # reconecta si el broker
volvió
        with psycopg.connect(DSN) as conexion:
            pendientes = conexion.execute(
                "SELECT id, payload FROM outbox WHERE estado = 'PENDIENTE' "
                "ORDER BY creado_en LIMIT 100").fetchall()
            for evento_id, payload in pendientes:
                canal.basic_publish(
                    exchange="", routing_key=COLA, mandatory=True,
                    body=json.dumps({"eventoId": str(evento_id), **payload}),
                    properties=pika.BasicProperties(delivery_mode=2,
message_id=str(evento_id)),
                )
                # Solo se llega acá si RabbitMQ confirmó que guardó el
mensaje
                conexion.execute(
                    "UPDATE outbox SET estado = 'PUBLICADO', publicado_en =
now() WHERE id = %s",
                    (evento_id,))
                conexion.commit()
    except (pika.exceptions.AMQPError, psycopg.OperationalError) as error:
        # Broker o base no disponibles: el evento sigue PENDIENTE y se
reintenta
        log.warning("No se pudo publicar, se reintenta: %s", error)
        canal = None
    time.sleep(1)
```

Tres detalles de este código son los que hacen que funcione: `confirm_delivery()` hace que `basic_publish` espere la confirmación del broker y lance una excepción si no la recibe; `pika.exceptions.AMQPError` es la clase base de los errores de pika (el `ConnectionError` estándar de Python no los atrapa); y la reconexión dentro del bucle permite retomar cuando el broker vuelve, porque una conexión cortada no se recupera sola.

4.3 Qué pasa cuando cae el broker

Es el escenario que pide la demostración sugerida por la cátedra ("simular la caída del broker y demostrar que un proceso posterior publica el evento sin perderlo"):

1. **Con RabbitMQ apagado:** el cliente envía `POST /viajes`; la API guarda viaje y evento en una transacción y responde `202`. El relay intenta publicar, recibe una excepción, no actualiza el evento y lo deja `PENDIENTE`. Esto se repite cada segundo durante toda la caída.
2. **Cuando RabbitMQ vuelve:** en la siguiente vuelta el relay se reconecta, publica el evento, recibe la confirmación y lo marca `PUBLICADO`. El worker lo toma y procesa el viaje. Ningún evento se perdió.

4.4 El caso límite: entrega al menos una vez

Si el relay publica con éxito pero se cae justo antes de marcar el evento como `PUBLICADO`, al reiniciar lo encuentra todavía `PENDIENTE` y lo publica otra vez. El Outbox garantiza que ningún evento se pierda, pero no que llegue una sola vez: la entrega es **al menos una vez**. Por eso el consumidor debe ser idempotente (AWS Prescriptive Guidance). Por el mismo motivo, conviene que haya un solo relay activo o que cada instancia "reclame" el evento con una actualización atómica antes de publicarlo.

4.5 Formas de implementar el relay

Estrategia	Cómo funciona	Ventaja	Desventaja
Polling publisher	Consulta la tabla `outbox` cada pocos segundos	Simple; solo necesita la base de datos	Consultas periódicas y demora igual al intervalo
Transaction log tailing (CDC)	Lee el registro de cambios de la base (en PostgreSQL, la replicación lógica del WAL, por ejemplo con Debezium)	Casi sin demora	Más infraestructura y configuración (Debezium Outbox Event Router)

4.6 Esquema de las tablas en PostgreSQL

La tabla `outbox` vive en la misma base que `viajes`. El índice parcial hace que la consulta del relay solo recorra los eventos pendientes, aunque la tabla tenga miles de eventos ya publicados. La tabla `eventos_procesados` es la que usa el consumidor para deduplicar (sección 5.3).

```
CREATE TABLE outbox (
    id           UUID PRIMARY KEY,
    tipo         TEXT NOT NULL,
    payload      JSONB NOT NULL,
    estado       TEXT NOT NULL DEFAULT 'PENDIENTE',
    creado_en    TIMESTAMPTZ NOT NULL DEFAULT now(),
    publicado_en TIMESTAMPTZ
);
CREATE INDEX outbox_pendientes ON outbox (creado_en) WHERE estado =
'PENDIENTE';

CREATE TABLE eventos_procesados (
    evento_id    UUID PRIMARY KEY,
    procesado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4.7 Aplicación en MongoDB (el motor del TPI)

El patrón no depende del motor, pero en MongoDB hay una diferencia importante: una transacción que escribe en dos colecciones (`viajes` y `outbox`) **no funciona en un MongoDB standalone**; requiere un *replica set*, que en desarrollo puede ser de un solo nodo (documentación de MongoDB). El `docker-compose.yml` actual de M2 levanta

MongoDB como standalone, así que para aplicar el patrón en el TPI habría que iniciarlo con `--replSet rs0`. La alternativa sin replica set es guardar el evento dentro del propio documento de negocio, porque MongoDB garantiza la atomicidad de la escritura de un único documento; microservices.io menciona esta variante para bases NoSQL.

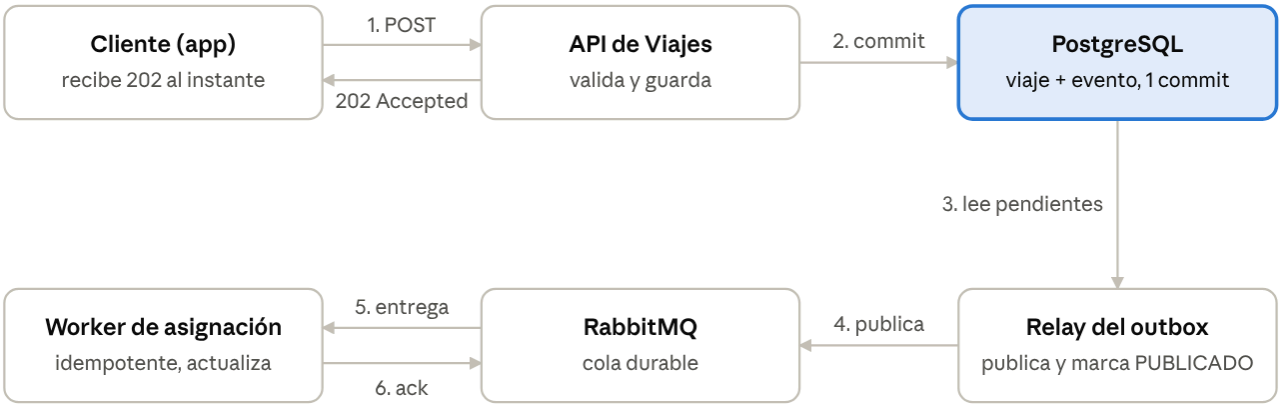
4.8 Beneficios y limitaciones

Beneficios	Limitaciones
Elimina la inconsistencia entre base y broker sin usar 2PC	Duplicados posibles: exige consumidores idempotentes
La API sigue funcionando con el broker caído	Demora entre el commit y la publicación (el intervalo del relay)
Los eventos se publican en el orden en que se registraron	Un proceso más para operar y una tabla que crece: hay que limpiar lo publicado
Registro auditable de los eventos emitidos (RNF-19)	Es fácil olvidar escribir en el outbox al agregar una operación: conviene encapsularlo en una función común

5. Integración de ambos temas

El outbox garantiza que el evento salga del servicio, y la cola garantiza que llegue al consumidor aunque este esté caído. Juntos cubren todo el recorrido: ninguna falla de un componente intermedio pierde el evento, solo lo demora.

El evento sale de la base solo si el viaje se guardó



flujo integrado · outbox + cola

1. El cliente envía `POST /viajes`. La API valida y, en una única transacción, guarda el viaje en estado `SOLICITADO` y el evento `ViajeSolicitado` en el outbox.
2. La API responde `202 Accepted` con un encabezado `Location` para consultar el estado.
3. El relay lee los eventos pendientes del outbox.
4. El relay los publica en RabbitMQ, espera la confirmación y los marca como publicados.
5. RabbitMQ entrega el mensaje al worker de asignación.
6. El worker procesa, actualiza el viaje a `CONDUCTOR_ASIGNADO`, registra el `eventoId` como procesado y envía el ack.

5.1 Comportamiento ante fallas

Falla	Qué pasa	Quién lo resuelve
La transacción de la base falla	No se guarda ni el viaje ni el evento; la API responde error	Atomicidad de la transacción (outbox)
RabbitMQ está caído	La API sigue respondiendo `202`; los eventos se acumulan en el outbox y se publican cuando vuelve	Relay con reintentos (outbox)
El relay se cae después de publicar y antes de marcar	Al reiniciar publica otra vez el mismo evento: llega un duplicado	Consumidor idempotente
El worker está detenido	Los mensajes esperan en la cola durable en estado Ready	Cola (tema 2)
El worker se cae a mitad del procesamiento	El mensaje pasa de Unacked a Ready y se reentrega	Ack manual (tema 2) + consumidor idempotente
Un mensaje falla siempre (dato inválido)	Se rechaza y se desvía a la Dead Letter Queue	DLQ (tema 2)

5.2 El punto de unión: al menos una vez + idempotencia

Tanto el relay como el broker reintentan para no perder nada, y todo lo que reintenta puede repetir. El sistema completo ofrece entrega **al menos una vez**. La entrega exactamente una vez entre sistemas distintos no se puede garantizar en general; lo que sí se puede garantizar es que el **efecto** ocurra una sola vez, con un consumidor idempotente (Idempotent Consumer). Es la exigencia del RNF-09 del TPI.

5.3 Consumidor idempotente

El worker registra cada `eventoId` en la tabla `eventos_procesados`, en la misma transacción que su propio cambio. Como `evento_id` es clave primaria, el `INSERT ... ON CONFLICT DO NOTHING` inserta la fila solo la primera vez; si el evento ya estaba, no inserta nada y el worker sabe que no debe repetir el efecto.

```
# worker.py
def procesar(canal, metodo, propiedades, cuerpo):
    mensaje = json.loads(cuerpo)
    conductor_id = buscar_conductor(mensaje["viajeId"])   # operación lenta,
sin efectos guardados
    try:
        with psycopg.connect(DSN) as conexion:            # una transacción:
COMMIT al salir
            nuevo = conexion.execute(
                "INSERT INTO eventos_procesados (evento_id) VALUES (%s) "
                "ON CONFLICT (evento_id) DO NOTHING",
                (mensaje["eventoId"],)).rowcount
            if nuevo == 1:                                # primera vez que
llega este evento
                conexion.execute(
                    "UPDATE viajes SET estado = 'CONDUCTOR_ASIGNADO',
conductor_id = %s "
                    "WHERE id = %s",
                    (conductor_id, mensaje["viajeId"]))
            # nuevo == 0: ya procesado, se confirma sin repetir el efecto
    except psycopg.Error:
        # Falla de la base: se devuelve el mensaje a la cola para reintentar
        canal.basic_nack(delivery_tag=metodo.delivery_tag, requeue=True)
        return
    canal.basic_ack(delivery_tag=metodo.delivery_tag)
```

Esta versión evita la condición de carrera de "consultar el estado y después actualizar". Si dos workers reciben el mismo evento a la vez, PostgreSQL hace esperar al segundo `INSERT` hasta que el primero confirma, y entonces no inserta nada: solo uno aplica el cambio. Si el worker se cae después del `COMMIT` y antes del ack, el mensaje se reentrega, el `INSERT` no inserta y se confirma sin asignar dos veces. Todas las consultas usan parámetros (`%s`) en lugar de armar el SQL con texto, lo que evita la inyección SQL.

6. Relación con el Trabajo Práctico Integrador

La plataforma de movilidad del TPI tiene nueve módulos que "colaborarán mediante APIs o eventos" y cada uno es dueño de sus datos (RNF-05). Hoy todos los módulos se comunican por REST; el TP2 incorpora RabbitMQ (RNF-10). Los dos temas aplican donde un módulo cambia su estado y otros módulos tienen que enterarse.

6.1 Requisitos del TPI vinculados

Requisito	Qué pide	Tema que lo atiende
RNF-10 Mensajería	RabbitMQ o alternativa para flujos asíncronos	Colas
RF-8.8 Consumo asíncrono	M8 procesa eventos desde una cola para no bloquear el flujo principal	Colas
RNF-09 Idempotencia	Los consumidores toleran reintentos sin duplicar efectos	Ambos (consumidor idempotente)
RF-9.10 Activación idempotente	Una reserva genera como máximo una solicitud de despacho	Outbox + consumidor idempotente
RNF-13 Resiliencia	Ante la caída de un servicio, comportamiento explícito y sin bloqueos indefinidos	Ambos
RNF-03 Contratos	Eventos documentados en un catálogo común	Ambos (formato del evento)
RNF-19 Auditoría	Trazabilidad de operaciones críticas	Outbox (registro de eventos emitidos)

6.2 Aplicación por módulo

Módulo	Proceso	Evento	Consumidores	Aplicación
M5 Solicitud y Despacho	Solicitud de viaje inmediata (RF-5.1, RF-5.3)	`ViajeSolicitado`	Worker de búsqueda de candidatos	La API responde `202` y la búsqueda de conductores corre en segundo plano. Outbox para que la solicitud guardada siempre dispare la búsqueda

Módulo	Proceso	Evento	Consumidores	Aplicación
M6 Viajes	Finalización del viaje (RF-6.5)	`ViajeFinalizado`	M7 (cobro), M8 (comprobante PDF)	El cambio de estado y el evento se guardan juntos: un viaje finalizado sin cobro sería una pérdida para la plataforma
M7 Tarifas y Pagos	Captura del pago (RF-7.3, RF-7.6)	`PagoCapturado`	M8 (comprobante y aviso)	Consumidor idempotente para no cobrar dos veces el mismo `ViajeFinalizado`
M8 Notificaciones	Avisos, QR y comprobantes (RF-8.1, RF-8.4, RF-8.8)	Consume los anteriores	—	Caso típico del tema 2: enviar un mail o generar un PDF es lento y no debe frenar el viaje
M9 Reservas	Activación de una reserva (RF-9.9, RF-9.10)	`ReservaActivada`	M5	Outbox para que la reserva pase a `ACTIVADA` y el evento salga juntos; M5 deduplica por `reservaId` para crear una sola solicitud
M2 Clientes (nuestro módulo)	Bloqueo y desbloqueo de cuenta (RF-2.6)	`ClienteBloqueado`, `ClienteDesbloqueado`	M5, M9	Hoy M5 y M9 consultan el estado por REST en cada pedido. Con el evento pueden mantener una copia local o invalidar su caché (RNF-11)
M2 Clientes	Calificación del conductor (RF-2.5)	`ConductorCalificado`	M3	M3 actualiza el promedio del conductor sin que M2 dependa de que M3 esté en línea

6.3 Cómo se aplicaría en el código de M2

En M2, el endpoint `crear_bloqueo` de `rutas/bloqueos.py` hoy hace un solo `insert_one` en la colección `bloqueos`. Con el patrón, haría dos escrituras dentro de una sesión con transacción de MongoDB, con la misma lógica de la sección 4.2: el bloqueo y el evento

`ClienteBloqueado` en una colección `outbox`. Un relay publicaría los eventos pendientes en un exchange `clientes` de RabbitMQ. Para que la transacción funcione, el servicio `mongodb` del `docker-compose.yml` de M2 tendría que arrancar como replica set de un nodo.

Los eventos entre módulos los acuerda el consorcio de grupos; según el README de M2, "por ahora el contrato no tiene eventos asíncronos". Las filas de M2 son una propuesta para el TP2.

7. Criterios de utilización

Una cola conviene cuando quien pide no necesita el resultado en el momento; el outbox conviene cuando perder o inventar un evento tiene un costo de negocio real. Si ninguna de las dos condiciones se cumple, la llamada REST directa es más simple y suficiente.

7.1 Colas de mensajes

Conviene usarla	No se recomienda
La operación tarda segundos o más (generar un PDF, enviar mails, buscar conductores)	El usuario necesita la respuesta para continuar (consultar el perfil, validar el estado de cuenta antes de reservar)
El consumidor puede estar caído o ser más lento que el productor	La operación es rápida y el sistema es chico: el broker agrega más complejidad que beneficio
Hay picos de tráfico que conviene absorber	Se necesita orden estricto global y un solo consumidor no alcanza
Varios módulos reaccionan al mismo hecho	Se requiere confirmar en el momento un cambio en dos servicios (conviene rediseñar o usar una Saga)

7.2 Transactional Outbox

Conviene usarlo	No se recomienda
Un cambio en la base debe producir sí o sí un evento (viaje finalizado → cobro)	El evento es solo informativo y perderlo ocasionalmente no afecta (métricas, logs)
Perder un evento o publicar uno que no corresponde rompe la consistencia entre módulos	El servicio no tiene base de datos propia o no publica eventos

Conviene usarlo	No se recomienda
La base soporta transacciones o escrituras atómicas de un documento	Ya se usa Event Sourcing: los eventos almacenados son la fuente de verdad y pueden publicarse directamente
Se necesita auditoría de los eventos emitidos	La plataforma ya ofrece captura de cambios (CDC) administrada que cumple la misma función

7.3 Comparación con alternativas

Alternativa	Complejidad	Rendimiento	Garantía de consistencia	Mantenibilidad
REST síncrono directo	Baja	El usuario espera toda la cadena	Si el destino falla, la operación falla	Alta: fácil de seguir
Cola sin outbox (guardar y publicar)	Media	Respuesta rápida	Eventos perdidos si se cae entre los dos pasos	Media
Cola + Transactional Outbox	Media-alta: relay, tabla extra, idempotencia	Respuesta rápida; demora de publicación igual al intervalo del relay	Alta: al menos una vez, sin pérdidas	Media: más piezas, pero cada una es simple
Transacción distribuida (2PC)	Alta	Bloqueos y latencia entre servicios	Alta, si todos los participantes la soportan	Baja: acopla los servicios; RabbitMQ no la soporta
Outbox con CDC (Debezium)	Alta: Kafka Connect o similar	Muy baja demora	Alta	Media: más infraestructura que operar

7.4 Costos a considerar

- **Infraestructura:** un contenedor más (RabbitMQ) y un proceso más (relay).
- **Almacenamiento:** la tabla `outbox` crece; lo publicado se borra o archiva después de un período, por ejemplo con una tarea programada diaria.

- **Desarrollo:** diseñar el formato de los eventos, el catálogo común (RNF-03) y consumidores idempotentes.
- **Operación:** monitorear la longitud de las colas, la DLQ y la cantidad de eventos pendientes en el outbox.

8. Implementación práctica

La demo es un repositorio independiente del TPI e implementa el flujo de la sección 5 con Python 3.13, FastAPI, PostgreSQL 16, RabbitMQ y Docker Compose. Usa PostgreSQL aunque el TPI use MongoDB (ver la aclaración de la sección 1). Se levanta con un solo comando y cada falla se provoca deteniendo un contenedor. La operación lenta del worker se simula con una pausa de algunos segundos, para poder observar cómo se acumulan los mensajes cuando el consumidor está detenido.

Repositorio: github.com/Lucasema/2026-01-investigacion. El README explica cómo instalarlo y ejecutarlo, describe paso a paso los siete escenarios de la demostración y cubre las 12 pruebas de integración con PostgreSQL y RabbitMQ reales. Las pruebas se ejecutan automáticamente con GitHub Actions en cada cambio.

8.1 Componentes

Componente	Responsabilidad
`api` (FastAPI)	`POST /viajes`: inserta viaje + evento en una transacción y responde `202` con `Location`. `GET /viajes/{id}`: estado actual
`relay`	Publica cada 1 segundo los eventos `PENDIENTE` del outbox, con publisher confirms, y los marca `PUBLICADO`
`worker`	Consume `ViajeSolicitado` con ack manual y `prefetch_count=1`, simula la búsqueda de conductor y registra el `eventoId` como procesado
`rabbitmq`	Cola durable `asignacion.viaje-solicitado` y su Dead Letter Queue; consola de administración para observar los mensajes Ready y Unacked
`postgres`	Tablas `viajes`, `outbox` y `eventos_procesados`

8.2 Escenarios y evidencias

#	Escenario	Evidencia esperada
1	Flujo normal	`POST` responde `202` en milisegundos; segundos después `GET` muestra `CONDUCTOR_ASIGNADO`
2	Worker detenido	Las solicitudes siguen respondiendo `202`; la consola de RabbitMQ muestra mensajes Ready acumulados; al reiniciar el worker se procesan todos
3	Worker detenido a mitad de una tarea	El mensaje pasa de Unacked a Ready y se procesa al reiniciar
4	RabbitMQ caído	La API sigue respondiendo `202`; el outbox acumula eventos `PENDIENTE`; al volver el broker el relay los publica y ninguno se pierde
5	Comparación sin outbox	La versión que guarda y publica directo pierde el evento cuando el broker está caído: el viaje queda en `SOLICITADO` para siempre
6	Mensaje duplicado	Se fuerza la republicación de un evento; el worker lo detecta por `eventoId` y no asigna dos veces
7	Mensaje inválido	El mensaje se rechaza y termina en la Dead Letter Queue

Las evidencias se registran con capturas de la consola de RabbitMQ, consultas a PostgreSQL, logs correlacionados por `viajeId` y las pruebas automatizadas con pytest, que cubren todos los escenarios.

9. Conclusiones

Las colas de mensajes desacoplan en el tiempo a quien pide de quien procesa: la respuesta es inmediata y la caída de un consumidor demora el trabajo en lugar de perderlo, gracias al ack manual y a las colas durables. Su punto débil es la publicación, porque guardar en la base y publicar en el broker son dos escrituras sin una transacción común.

El Transactional Outbox cierra ese hueco con una transacción local y un relay, sin recurrir a 2PC. El costo es aceptar la entrega al menos una vez y, por lo tanto, diseñar consumidores idempotentes, que es lo que el TPI exige en el RNF-09.

No son patrones para todo: agregan infraestructura, consistencia eventual y más piezas para depurar. En el TPI se justifican en los flujos donde un hecho de negocio debe llegar a otros módulos sin falta (viaje finalizado, reserva activada, cuenta bloqueada) y no en las consultas que el usuario necesita al instante.

10. Bibliografía y fuentes

- Richardson, C. Pattern: Transactional outbox. microservices.io.
- Richardson, C. Pattern: Idempotent Consumer. microservices.io.
- Richardson, C. (2018). *Microservices Patterns*. Manning. Cap. 3, "Interprocess communication in a microservice architecture".
- Amazon Web Services. Transactional outbox pattern. AWS Prescriptive Guidance.
- RabbitMQ. Consumer Acknowledgements and Publisher Confirms.
- RabbitMQ. Dead Letter Exchanges.
- Microsoft. Asynchronous Request-Reply pattern. Azure Architecture Center.
- Microsoft. Queue-Based Load Leveling pattern. Azure Architecture Center.
- Microsoft. Competing Consumers pattern. Azure Architecture Center.
- PostgreSQL. Transactions e INSERT … ON CONFLICT. Documentación oficial.
- Psycopg. Transactions management. Documentación de psycopg 3.
- MongoDB. Production Considerations (Transactions).
- Debezium. Outbox Event Router.
- Hohpe, G. y Woolf, B. (2003). *Enterprise Integration Patterns*. Addison-Wesley. Patrones Message Channel, Guaranteed Delivery e Idempotent Receiver.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Cap. 11, "Stream Processing" (escrituras duales y captura de cambios).
- Cátedra Desarrollo de Software UTN FRRe. Enunciado del TPI 2026.